# A minimal tamper-evident layer for multi-agent ledgers

**Track 2 — Trust Infrastructure** AI x Epistemics Research Sprint, 13–15 November 2026

> **What minimal tamper-evident layer is missing from today's AI observability > standards, so that records of what agents did and agreed can be audited and > trusted?**

All numbers below come from `python3 eval.py`, `python3 eval_chain.py` and `python3 sprint2026_anchor.py` (the last added 2026-11-14, inside the sprint window). Nothing here is estimated, and the limits are stated rather than buried.

---

## 1. The question, narrowed

Track 2 asks for a *minimal* layer. We read that word as a constraint, not a modesty qualifier: the thing has to be small enough that a team shipping an agent framework today would actually adopt it, because anything requiring key management, a service, or a new runtime gets postponed forever.

So the target is narrow on purpose:

> Can a log that is handed over later be checked — cheaply, offline, by someone > who does not trust the writer — for edit, deletion, insertion, reordering and > truncation?

Note what this deliberately excludes: *authenticity* (who wrote it). That is a signature problem, and every answer to it costs key management. We do not claim to solve it, and we say so in section 5.

## 2. What we added

`swarmbox/chain.py` — 118 lines, standard library only. It sits on top of the existing append-only ledger and adds exactly two fields per record plus one external anchor:

| Field | Role |
|---|---|
| `_prev` | digest of the preceding record |
| `_h` | digest of `(prev, canonical(this record))` |
| tail anchor | digest of the final record, published separately at write time |

The practical consequence is that integrity becomes *localisable*: a verifier does not merely learn "something is wrong", it learns which index is wrong and which kind of wrong it is. That difference decides whether the log can be repaired or has to be discarded.

Two design details turned out to matter more than expected:

- **Canonical bytes.** Records are hashed after `sort_keys=True` JSON with no incidental whitespace, so the digest survives a round trip through another language or platform. Reserialising after the fact otherwise breaks every hash.
- **Opt-in chaining.** `Ledger(chained=True)` produces chained receipts; the default is unchanged. Previously published results and their digests — and the 235 assertions in `eval.py` — are byte-for-byte identical before and after.

## 3. What is tested

`eval_chain.py` adds **35 assertions, 0 failures**, grouped as follows. `sprint2026_anchor.py` (added 2026-11-14, the sprint window) adds **7 assertions, 0 failures** covering the external-anchor mechanism — see section 5.

- **Backwards compatibility** — default runs carry no new fields; the whole-file digest is still a sha256.
- **Well-formedness** — every chained record carries both link fields, the first links to genesis, and chaining records the same event count as a plain run.
- **The experiment does not change** — a chained and an unchained run with the same seed produce identical Gini of holdings.
- **Attacks** — editing a field mid-log, deleting a record, duplicating a record in place, swapping two adjacent records, and stripping records from the head are each detected, localised by index, and labelled with the failure mode.
- **Determinism** — the same seed reproduces a byte-identical chained ledger and a stable tail digest; a different seed does not.
- **Round trip** — chained receipts written to disk read back and verify against their anchor.

Total for the repository: **277 assertions, 0 failures** (235 + 35 + 7).

## 4. The boundary we did not paper over

A hash chain detects edits, deletions, insertions, reordering and head deletions. It **does not detect a silent tail truncation** on its own: every record left in a truncated file still links up perfectly, so an untampered-looking log may simply be shorter than the truth. This is a property of the primitive, not of our implementation, and it is easy to miss if a team adopts "we hash-chained it" as a slogan.

We handle it the only way it can be handled: the tail digest must be anchored somewhere the writer cannot later rewrite. We made this a test rather than a footnote — `verify_chain(rows)` passes on truncated input, while `verify_chain(rows, expected_tail=anchor)` fails with a distinct reason. Both behaviours are asserted, so the limitation cannot silently regress into a false sense of security.

## 5. Sprint addition (2026-11-14, in window): the anchor, implemented

Section 4 states the gap — a hash chain cannot see a silent tail truncation. During this sprint we turned that footnote into running code and tests. The new module `sprint2026_anchor.py` (added 2026-11-14) implements the external anchor and asserts the attacks it defeats:

- **Publish the anchor.** The tail digest (`summarise(rows)["tail"]`) is written to an *external* registry the reader fetches on their own — simulated here as a separate store with its own `source` tag, never colocated with the log.
- **Clean handoff verifies** against the published anchor (pass).
- **Truncation is caught** — drop the last 3 records and `verify_chain(..., expected_tail=anchor)` fails with a distinct *"tail digest does not match the anchor"* reason.
- **The gap is still real without the anchor** — the *same* truncated log verifies fine when no anchor is supplied. This is asserted too, so the report does not quietly overclaim; the anchor is the only thing that closes the gap.
- **Forged append** — an attacker appends a record with a *valid internal* hash (chained from the last anchored row) *after* the anchored tail. The in-log chain still links, but the recomputed tail no longer

equals the anchor, so it is caught. This is the realistic "I added a benign-looking line after shipping" attack, and it is exactly what a slogan of "we hash-chained it" would miss.

- **Tampered anchor** — a wrong digest in the registry is rejected by an honest reader, proving the anchor itself is part of what must be trusted.
- **Independent-reader audit** and **determinism** (identical logs → identical anchor) round out the suite.

New assertions: **7 passed, 0 failed**. The mechanism is 38 lines, standard library only, offline, deterministic — same discipline as the rest of the project, and directly answers Track 2's prompt about what minimal tamper-evident layer is *missing* from today's observability standards.

## 6. Limitations — read these before citing us

- **Integrity, not authenticity.** Anyone holding the log can re-chain it into a self-consistent fake. A chained log proves nothing about authorship without a signature or HMAC key, and we ship neither.
- **The anchor is external.** All truncation detection depends on somebody having written the tail digest down somewhere else. If that record is stored next to the log, an adversary who can edit one can edit both.
- **No revocation, no non-repudiation, no timestamps.** Those belong to a real transparency log with an append-only store and external witnesses.
- **Single-threaded assumption.** Concurrent writers need per-writer chains or a lock; nothing here serialises them.
- **Small evidence base.** The `swarmbox` ledger is our own testbed. We have not run this against a production agent framework, and "trust infrastructure" claims should survive contact with one before anyone depends on them.

## 7. Dual-Use / Ethical Considerations

A tamper-evident ledger is a double-edged instrument, and we state the edges.

- **Forgery, made easier to ship.** Because the layer is signature-free, anyone who holds a log can re-chain it into a self-consistent fake. We do *not* make forgery harder; we make *detection of post-hoc edits to a log you already trusted* cheaper. The danger is that a team treats "chained" as "proven" and leans on it for accountability it cannot provide. Our report names this explicitly (section 6) rather than letting the framing imply authenticity.
- **Surveillance via audit trails.** Fine-grained, tamper-evident agent logs make it cheap to reconstruct *who decided what*, which is useful for safety and incident review but also for workplace surveillance. The layer should audit actions, not justify monitoring people without consent or due process.
- **False confidence in high-stakes settings.** Presenting this as "trust infrastructure" near real financial or safety systems, before it has met a production agent framework, is the misuse we most want to pre-empt. The "small evidence base" limitation (section 6) is the guardrail against that.
- **What we recommend.** Anchor digests in a write-once location independent of the log; pair the layer with real authentication where authorship matters; and treat a passing verification as "this file is byte-stable since the anchor was set", never as "this content is true".

## 8. Reproduce

```
python3 eval.py              # 235 passed, 0 failed  (unchanged, no regression)
python3 eval_chain.py        #  35 passed, 0 failed  (the tamper-evident layer)
python3 sprint2026_anchor.py#   7 passed, 0 failed  (2026-11-14 anchor work)
```

To produce and verify a chained ledger yourself:

```
python3 - <<'PY'
from swarmbox import run
from swarmbox.chain import summarise, verify_chain
import copy

r = run(n=6, rounds=4, seed=7, ledger_chained=True)
rows = r.ledger.rows
print("tail:", summarise(rows)["tail"])

tampered = copy.deepcopy(rows)
tampered[len(tampered) // 2]["round"] = 99999
print("after edit ->", verify_chain(tampered))
PY
```

Expected: the clean log verifies; the edited one fails at the index you touched.

## 9. Lineage

This is an increment on `swarmbox`, a Track 1 testbed built for the earlier Swarm Alignment Hackathon. That project's contribution was making refusal *inescapable* rather than merely *blocked*; this one applies the same instinct to the audit trail — make tampering *visible at a specific index* rather than merely *discouraged*. Both are standard library only, deterministic, and offline.